

Test Plan Inventory Module (Sauce Demo Application)

Disclaimer

- All the Section mentioned below are mandatory and should not be deleted.
- In case if any section is not applicable for your project, mark it as NA
- This is a standard template which cover important and mandatory section of the plan, please do not restrict yourself only with these and based on Requirement feel free to add more section or sub section in your Test Plan.

Reference Documents

1	Version	Document Type	Document Name / Url
2	1	BRD	Observed Sauce Demo Application behavior and functionalities
3	1	UI Mockup/Design document [if available.]	Observed live Sauce Demo Application UI (Production environment)
4	N/A	Project Wiki Link	N/A

Final Design Review and Approval

1	Name (Alias@)	Role	Approver / Reviewer	Approval / Review Date / Comments
2		UX Designer	Approver	Approved
3		SDM	Approver	Approved
4		Dev	Approver	Approved
5		PM	Approver	Approved
6		QA	Reviewer	Reviewed

GEN AI CONSUMPTION AND SAVE [Overall Save ~ X days/hours]

Phase	Usage	Impact [Save in Days]	Tool Used
Requirement Analysis Phase	Manually understood application flow and features by exploring the Sauce Demo application	0	Manual Exploration
Test Planning Phase	Used AI assistance for preparing test cases, test scenarios, and test plan documentation	2-3 days saved	ChatGPT
Test Execution Phase	Used Gen AI for execution approach guidance and validation ideas	1 day	Selenium WebDriver + TestNG
Automation	Used Gen AI for Selenium framework structure, POM design, and debugging support	3-5 days saved	ChatGPT
Report Generation	Not yet implemented	N/A	

1.Introduction

The Inventory module is accessible after a successful login using valid user credentials. The Sauce Demo application provides different user types to test various scenarios, where certain users may experience UI-related issues such as incorrect product images or pricing inconsistencies. However, the standard user is able to access and use the Inventory module without such issues. The Inventory page displays a list of available products along with product images, product names, descriptions, and prices. Users can perform actions such as adding products to the cart, removing products from the cart, and viewing individual product details pages.

1.1 Acceptance Criteria

- Valid users should be able to successfully access the Inventory module.
- Products should be displayed with correct product information.
- Users should be able to add and remove products from the cart.
- Users should be able to navigate to the product details page.
- Locked users should not be able to access the application.
- Different user types should display expected application behaviours and restrictions.
- Users should be able to proceed with shopping and checkout functionalities based on user permissions.

2.Scope

2.1 In Scope -

Form Factor Coverage -

Form Factor	Applicable?
Android	☐
iOS	☐
mWeb	☐
Desktop	☒

2.2 Out of Scope

- Mobile application testing
- Android application testing
- iOS application testing
- Performance testing
- Security testing
- Database testing
- API testing (Public APIs are not available for the Sauce Demo application)

3. Test Strategy

3.1 TEST TYPE CHECKLIST

Test Type	Applicable ?
Functional Testing	✓
Monkey Testing	✓
Security Testing	✓
Performance Testing Manual	✓
Latency Testing	✓
Font/Typographyic Testing	✓
Regression Testing	✓
Compatibility Testing	✓
Low Network Testing	✓
Bug Bash	✓

3.2 TEST STRATEGY DETAIL - Inventory Module

1 Functional Testing

Functional testing will be performed to verify that all Inventory page functionalities work according to the expected application behaviour and requirements.

Areas Covered

- Product display validation
- Product name validation
- Product description validation
- Product price validation
- Product image validation
- Add to Cart functionality
- Remove from Cart functionality
- Product details page navigation

2. Monkey Testing

Monkey testing will be performed by randomly interacting with Inventory page elements without predefined test steps to identify unexpected behaviour or instability.

Areas Covered

- Random Add/Remove actions
- Rapid multiple clicks
- Random product navigation
- Browser refresh actions
- Random cart operations

User Types Covered

- problem_user
- performance_glitch_user

3.Security Testing

Basic security testing will be performed to validate user access restrictions and authentication behavior related to the Inventory module.

Areas Covered

- Unauthorized Inventory page access
- Locked user access validation
- Session access validation

4. Manual Performance Testing

Basic manual performance testing will be performed to observe application responsiveness and delay behavior during Inventory page interaction.

Areas Covered

- Inventory page loading
- Product image loading
- Add to Cart response time
- Remove button response time
- Product navigation response time

User Types Covered

- performance_glitch_user

5. Latency Testing

Latency testing will be performed to observe delays between user actions and system responses within the Inventory module.

Areas Covered

- Product page navigation delay
- Add to Cart response delay
- Remove button response delay
- Inventory page load response

6. Font / Typography Testing

Font and typography testing will be performed to validate UI readability and text consistency on the Inventory page.

Areas Covered

- Product name visibility
- Product description readability
- Price visibility
- Header visibility
- Button label readability
- Text alignment consistency

7. Regression Testing

Regression testing will be performed after framework updates or Inventory module changes to ensure existing functionalities continue working correctly.

Areas Covered

- Product display
- Cart functionality
- Product navigation
- UI validations

8. Compatibility Testing

Compatibility testing will be performed to verify consistent Inventory page behaviour across supported browsers.

Browsers Covered

- Google Chrome
- Microsoft Edge

Areas Covered

- UI rendering
- Navigation behaviour
- Button functionality
- Product display consistency

9. Low Network Testing

Low network testing will be performed to observe Inventory page behaviour under slow or unstable internet conditions.

Areas Covered

- Product image loading delay
- Inventory page loading delay
- Cart update delay
- Navigation responsiveness

10. Bug Bash / Exploratory Testing

Bug Bash and exploratory testing will be performed to identify hidden defects and unexpected behavior through random feature exploration.

Areas Covered

- Random Inventory interactions
- Product navigation behaviour
- Add/Remove cart actions
- UI inconsistency observation
- End-to-end Inventory workflow validation

3.3 TEST COVERAGE-

OS Coverage -

Priority	Operating System	Browser
P0	Windows 11	Google Chrome
P1	Windows 11	Microsoft Edge
P2	Windows 11	Firefox

Browser	Screen Size	Model
Google Chrome	1920 × 1080	ASUS VivoBook X421EAY / K413EA
Microsoft Edge	1920 × 1080	ASUS VivoBook X421EAY / K413EA
Firefox	1920 × 1080	ASUS VivoBook X421EAY / K413EA

3.2 Automation Plan

UI Automation

UI automation will be implemented for the Inventory module using Selenium WebDriver with Java. The automation framework follows the Page Object Model (POM) design pattern to maintain reusable, scalable, and maintainable test scripts.

The framework uses Git for source code management and version control. Jenkins will be used for automated build execution after code changes and integration activities.

Data-Driven Testing (DDT) approach will be implemented to execute a single test method with multiple test data sets. The framework also includes screenshot capture functionality for failed test cases to support debugging and defect analysis.

Call Outs if any

- Screenshot on failure implemented
- Cross-browser support available
- DDT implemented
- Jenkins integration available
- Framework is scalable and reusable

Details of the Framework used

- Automation framework is developed using Selenium WebDriver with Java.
- Page Object Model (POM) design pattern is used for better maintainability and reusability.
- Test execution is managed using TestNG.
- Maven is used for dependency and build management.
- Git is used for source code management.
- Jenkins is integrated for automated build and test execution.
- Data-Driven Testing (DDT) approach is implemented to execute test cases with multiple test data sets.
- Screenshot capture functionality is implemented for failed test cases.
- Cross-browser testing support is available for:
- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Assertions are used for validation and verification of expected results.

Details of Package used

- Selenium WebDriver
- TestNG
- Maven
- WebDriverManager
- Apache POI (for Data-Driven Testing if Excel data is used)
- Extent Reports / Reporting library (if implemented)
- Java Utility Packages
- Selenium Support Package

Functionalities Covered

The following Inventory module functionalities will be covered through automation testing:

- Product display validation
- Product name validation
- Product description validation
- Product price validation
- Product image validation
- Add to Cart functionality
- Remove from Cart functionality
- Cart badge count validation
- Product details page navigation
- Inventory page UI validation
- Cross-browser validation
- User-based behavior validation (standard_user, problem_user, performance_glitch_use)

TC eligible for Automation

Test Scenario / Validation	Testing Type
Verify standard_user can access Inventory page successfully	Functional Testing
Verify standard_user can view all products in Inventory list	Functional Testing
Verify product names are displayed correctly	UI Testing / Functional Testing
Verify product descriptions are displayed correctly	UI Testing
Verify product prices are displayed correctly with \$ symbol	UI Testing
Verify product images are displayed correctly	UI Testing
Verify standard_user can add products to cart	Functional Testing
Verify standard_user can remove products from cart	Functional Testing
Verify cart badge count updates correctly	Functional Testing
Verify user can navigate to product details page	Functional Testing
Verify locked_out_user cannot access Inventory page	Security Testing / Functional Testing
Verify problem_user displays incorrect product images	Functional Testing / Exploratory Testing
Verify problem_user can still add products to cart	Functional Testing
Verify problem_user can view product descriptions and prices	Functional Testing
Verify problem_user faces issues during checkout flow	Exploratory Testing
Verify problem_user faces inconsistent click behavior	Monkey Testing / Exploratory Testing
Verify performance_glitch_user experiences delayed navigation	Performance Testing (Manual) / Latency Testing
Verify delayed response during Add to Cart action	Latency Testing
Verify Inventory page loading under slow network	Low Network Testing
Verify Add to Cart button changes to Remove button after click	Functional Testing / UI Testing
Verify Remove button style is displayed correctly	UI Testing
Verify Add to Cart button style is displayed correctly	UI Testing
Verify product names are displayed in dark green color	UI Testing
Verify Swag Labs header is displayed at top-center	UI Testing
Verify cart icon is displayed at top-right corner	UI Testing
Verify Inventory page behavior across browsers	Compatibility Testing
Perform random add/remove and navigation actions	Monkey Testing
Perform random exploratory validation across Inventory module	Bug Bash / Exploratory Testing
Verify no application crash during repeated actions	Stability Testing / Monkey Testing

3.3 Test Environments

Component	Details
Application Under Test	Sauce Demo
Environment Type	Production (Live Environment)
Operating System	Windows 11
Browsers	Google Chrome, Mozilla Firefox, Microsoft Edge
Screen Resolution	1920 × 1080
Test Type	Manual Testing & UI Automation Testing

3.4 Test Data

The following types of test data will be used during Inventory module testing:

- User data
- Product data
- Cart data
- Sorting data
- UI validation data
- Browser data

Test Accounts	What You Test
standard_user	Normal Inventory functionality
locked_out_user	Access restriction
problem_user	UI inconsistencies
performance_glitch_user	Delayed response behavior

3.5 Test scenarios

- Verify successful navigation to Inventory page after valid login.
- Verify all products are displayed correctly in the Inventory list.
- Verify product names are displayed correctly.
- Verify product descriptions are displayed correctly.
- Verify product prices are displayed correctly with currency symbol.
- Verify product images are displayed correctly.
- Verify Add to Cart functionality for products.
- Verify Remove from Cart functionality.
- Verify cart badge count updates correctly after cart operations.
- Verify navigation to product details page.
- Verify Inventory page UI elements are displayed correctly.
- Verify Swag Labs header visibility on Inventory page.
- Verify cart icon visibility and functionality.
- Verify product sorting functionality.
- Verify Inventory module behaviour for different user types.
- Verify Inventory page responsiveness under low network conditions.
- Verify delayed navigation behaviour for performance_glitch_user.
- Verify inconsistent UI behaviour for problem_user.
- Verify cross-browser compatibility of Inventory page.
- Verify Inventory module stability through exploratory and random testing.

3.2 Defects

Any defects identified during testing will be documented and tracked with the following details:

- Defect ID:
- Test Scenario ID:
- Test Case ID:
- Defect Summary:
- Steps to Reproduce:
- Expected Result:
- Actual Result:
- Severity:
- Priority:
- Reported By:
- Assigned To / Developer:
- Browser Details:
- Environment Details:
- Defect Screenshot / Evidence:
- Defect Status:

4.Assumptions and Dependencies

Assumptions

- The Sauce Demo production application is available and accessible during testing activities.
- Stable internet connectivity is available throughout test execution.
- Required system resources and electricity supply are available during testing.
- Supported browsers are properly installed and functioning.
- Test user accounts are active and accessible.
- The application environment remains stable during testing.
- Required automation tools and dependencies are properly configured.

Dependencies

- Internet connectivity
- Sauce Demo application availability
- Browser availability:
 - Google Chrome
 - Mozilla Firefox
 - Microsoft Edge
- Automation framework setup
- Test account availability
- System hardware and operating system availability

5.Risks and Contingencies

Risk	Impact	Mitigation
Internet connectivity issue	Test execution interruption	Ensure stable internet connection before execution
Sauce Demo application downtime	Testing cannot be performed	Reattempt testing after application availability
Browser compatibility issue	UI/functionality may behave differently	Execute testing across multiple supported browsers
Slow application response	Delayed test execution	Use proper waits and synchronization methods
Test data/account issue	Certain scenarios cannot be executed	Maintain valid and reusable test accounts
Automation script failure due to UI changes	Automation execution failure	Update locators and maintain reusable framework structure
System performance issue	Slow execution and instability	Close unnecessary applications during execution
Unstable behavior from problem_user or performance_glitch_user	False defect reporting possibility	Validate expected behavior before logging defects

6. Test Deliverables

S.No.	Deliverable Name	Author	Reviewer
1	Test Strategy and Test Plan	QAE@	QA Team, QAM, Dev Team/SDM
2	Test Cases	QAE@	QA Team/Dev Team/PM
3	Test Data	QAE@/PM@	QAM/SDE/SDM/PM/TPM
4	API Testing & IN-Sprint planned automation	QAE@	
5	First Test pass	QAE@	
6	Daily status report(including bug reports)	QAE@	QAM/SDM/TPM/PM/SDE/Clients QA if any
7	Defect Fix by Dev	SDE@	
8	Production Sanity	QAE@	
9	Defect Validation	QAE@	
10	Sanity & Sign-Off	QAE@	QAM/SDM/TPM/PM/SDE
11	Create sanity cases if required and update on the Intake	QAE@	
12	Share Quality metric	QAE@	QAM/SDM/TPM/PM/SDE
13	Client QA Sign Off		

7.Dev Build Plan

Build Version	ETA	Notes
Live Production Build	N/A	Public Sauce Demo application

8. QA Acceptance/Entry Criteria

This section should be pasted in the Status Mail.

Criteria	Status	Comment and ETA
Developer to Execute BVT Cases before build drop BVT List - here	Not Applicable	Sauce Demo is a production demo application; no separate build/BVT process exists
Developer to document and share all Code Review feedback in Quip, with descriptions of the associated functional changes and coverage	Not Applicable	Not Applicable for this Practice project
All the Open Questions are addressed	Not Applicable	Not Applicable for this Practice project
Weblab detail with access in place	Not Applicable	Not Applicable for this Practice project

9. QA Exit Criteria

This section will add in Intake Template as a mandatory field to be added before closing

Task	Status	Comment and ETA
100 % Test cases Executed.	Planned	All defined Inventory module test cases will be executed during test cycle
100% Test Strategy is covered and completed	Planned	Inventory module test strategy will be fully covered during test execution
All open issue are triaged and got alignment to differ.	Planned	Open issues will be triaged with stakeholders during test execution phase
Planned Automation completed	Planned	Automation activities will be completed as per defined scope
All the Open Bugs are triaged and have Product alignment	N/A	
Client Sign off are in place (If Applicable)	N/A	
Test Rail is updated	N/A	

10. Open Questions

Mention all the Open questions present during the design/development/review & execution phases.

- Parent sim containing all the issues
- Track any outstanding issues, owner for follow up and status. This list typically represents QA questions that are not answered yet.

11. Effort Estimation / person

Task Name	Ownership	Start Time	Finish Time	Status	Comments(If any)
Requirement Understanding	QA	11-May-2026	11-May-2026	Completed	Inventory module requirements were understood by exploring the Sauce Demo application.
Test Planning	QA	11-May-2026	13-May-2026	Completed	
Test cases writing	QA	11-May-2026	12-May-2026	Completed	
Test plan Review	QA	13-May-2026	13-May-2026	Completed	Test plan reviewed for Inventory module coverage and scope
Test Data collection if required	QA	12-May-2026	12-May-2026	Completed	Required Inventory module test data identified and documented
Planned buffer	QA	12-May-2026	12-May-2026	Planned	Reserved for unexpected delays during execution
First Pass completion	QA	12-May-2026	12-May-2026	Planned	Initial execution cycle of all planned Inventory module test cases will be performed during execution phase
Bug creation, Retest the bug and Triage	QA	13-May-2026	13-May-2026	Planned	will be followed during execution phase
Retest the failed cases and Sanity	QA	13-May-2026	13-May-2026	Planned	Failed scenarios will be retested and sanity testing will be performed during execution
Any test planned to test in prod	QA			Not Applicable	Separate production testing is not applicable for Sauce Demo practice project

QA LEAD SIGN-OFF

Prepared By:

Name: QA Lead

Date: May 13, 2026

Signature: Sumreen Banu H

Reviewed By:

QA Manager: _____

Development Manager: _____

Product Manager: _____

Final Approval:

SDM: _____

Date: _____